Research Progress Report

Reproducing "MLLMs Know Where to Look: Training-Free Perception of Small Visual Details with Multimodal LLMs"

Original Paper: Zhang et al., ICLR 2025
Repository: https://github.com/saccharomycetes/mllms_know
Report Date: March 30, 2026
Environment: Google Colab (T4 GPU, 15 GB VRAM)

Executive Summary

This report documents our progress in reproducing the ViCrop methods from Zhang et al. (ICLR 2025). We have successfully established the full pipeline, applied 4-bit quantization to work around GPU memory constraints, and run the `rel_att` and `grad_att` methods on both LLaVA-1.5 and InstructBLIP across subsets of TextVQA. On a 250-sample run with LLaVA-1.5, `rel_att` improved crop accuracy from 42.12% to 53.16%, qualitatively consistent with the paper's reported gains.

Contents

1 Overview and Objectives **2**

2 Theoretical Background **2**
2.1 The Perception Limitation Problem . . . 2
2.2 Relative Attention . . . 2
2.3 Gradient-Weighted Attention . . . 3
2.4 Input Gradient (pure-grad) . . . 3
2.5 Sliding-Window Bounding Box Selection . . . 3

3 Pipeline Architecture **3**
3.1 Full System Pipeline . . . 3
3.2 Attention Computation Detail . . . 4

4 Implementation Progress **4**
4.1 Session 1 – AIFP01: Manual Pipeline Exploration . . . 4
4.2 Session 2 – AIFP02: rel_att on LLaVA-1.5 . . . 5
4.3 Session 3 – AIFP02b: grad_att on LLaVA-1.5 . . . 5
4.4 Session 4 – AIFP03ab: InstructBLIP . . . 6

5 Results Summary **7**

6 Hardware Constraints and Workarounds **7**
6.1 Environment . . . 7
6.2 Workaround 1: 4-Bit NF4 Quantisation . . . 7
6.3 Workaround 2: Subset-Based Evaluation . . . 8
6.4 Workaround 3: Qwen2.5-VL Import Guard . . . 8
6.5 Workaround 4: BLIP bfloat16 → float16 . . . 8
6.6 Workaround 5: Q-Former Token Index Fix . . . 8

7 Known Issues and Open Questions **9**

8 Next Steps **9**

9 Conclusion **9**

A Experiment Log **10**

Overview and Objectives

This report summarises our ongoing effort to reproduce the core results of Zhang et al. (2025), "MLLMs Know Where to Look: Training-Free Perception of Small Visual Details with Multimodal LLMs," published at ICLR 2025. The paper makes two central contributions: (1) a systematic demonstration that MLLMs suffer from a *perception* bias against small visual objects rather than a localisation bias; and (2) a family of training-free visual cropping methods—collectively called **ViCrop**—that leverage the model's own attention maps and gradients to automatically zoom into the relevant image region before answering a question.

Our objectives are threefold. First, we aim to verify that the pipeline runs end-to-end on a resource-constrained environment. Second, we aim to reproduce the quantitative improvement trend shown in Table 2 of the paper, particularly on TextVQA. Third, we document the engineering challenges encountered and the workarounds we developed, so that any future collaborator can continue from where we stopped.

Theoretical Background

The Perception Limitation Problem

A fundamental assumption underpinning vision-language model evaluation is that model accuracy should be roughly size-invariant: if a model can identify a bird in a large crop, it should also identify it when the bird occupies a small fraction of the image. Zhang et al. show this assumption is violated. Across BLIP-2, InstructBLIP, LLaVA-1.5, and Qwen-VL, accuracy on TextVQA drops 7–24 percentage points when moving from bounding boxes that are large ($S \geq 0.05$) to small ($S < 0.005$), where $S = A_{\text{bb}}/A_{\text{total}}$ is the relative area of the answer bounding box.

Crucially, the authors rule out *localisation* as the cause: by computing **attention ratio**—the ratio of relative attention inside the ground-truth bounding box to the average across all equally-sized boxes—they show the ratio exceeds 1 in almost every layer and for both correctly and incorrectly answered questions. In other words, *the model knows where to look, but cannot resolve the fine detail it finds there.*

Relative Attention

To isolate semantically meaningful attention from global "register" tokens (which transformers use to aggregate background information), the paper normalises the answer-to-image attention by its value under a generic prompt q' = "Write a general description of the image.":

$$A_{\text{rel}}(x, q) = \frac{A_{si}(x, q)}{A_{si}(x, q')} \tag{1}$$

where $A_{si}(x, q) \in \mathbb{R}^{L \times L_c \times 1 \times N^2}$ is the *answer-to-image attention* computed as the tensor product of the answer-to-token attention $\hat{A}_{st}(x, q)$ and the token-to-image attention $\hat{A}_{ti}(x)$.

Gradient-Weighted Attention

To avoid the second forward pass required by relative attention, the **grad-att** variant replaces the generic-prompt normalisation with gradient-weighting:

$$v(x,q) = \log \operatorname{softmax}\big(z(x,q)\big)_{t^*}, \quad t^* = \arg\max_t z_t \tag{2}$$

$$\tilde{A}_{st}(x,q) = A_{st}(x,q) \odot \sigma\big(\nabla_{A_{st}} v(x,q)\big) \tag{3}$$

where $\sigma(w) = \max(0, w)$ removes negative gradients (locations that would *reduce* model confidence). The token-to-image attention is weighted identically. The combined importance map is used for sliding-window crop selection.

Input Gradient (pure-grad)

The third variant bypasses attention entirely and computes the L2-norm of the gradient with respect to the input image:

$$G(x,q) = \|\nabla_x v(x,q)\|_2 \tag{4}$$

A Gaussian high-pass filter followed by a median filter and edge-based binary mask suppresses spurious gradients in flat-colour regions (e.g. blue sky).

Sliding-Window Bounding Box Selection

All three methods produce an $N \times N$ importance map (where N is the ViT patch grid: 24 for LLaVA-1.5, 16 for InstructBLIP). A sliding window of fractional sizes $\{1, 1.2, 1.4, 1.6, 1.8, 2.0\} \times N$ is scanned across the map and the window with the highest internal sum is selected. A secondary criterion—maximum deviation from neighbouring window sums—prevents degenerate windows that encompass too little or too much context. The selected window is cropped, resized to the MLLM's input resolution, and prepended as an additional image to the token sequence.

Pipeline Architecture

Full System Pipeline

The figure below illustrates the end-to-end ViCrop pipeline as applied to LLaVA-1.5 (MLP connector) and InstructBLIP (Transformer Q-Former connector).

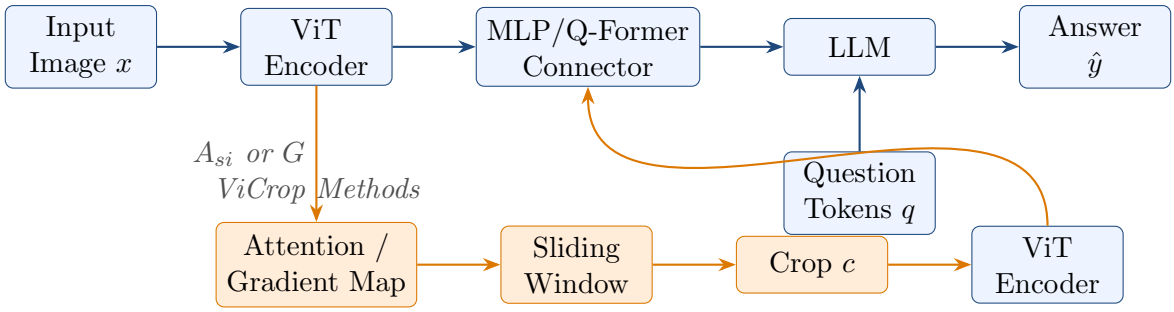

Figure 1: End-to-end ViCrop pipeline. The original image-question pair is processed normally (top row). In parallel, an importance map is extracted from the model's internal states (bottom row), used to select a tight crop c, which is then encoded and *concatenated* to the image tokens before passing through the LLM.

Attention Computation Detail

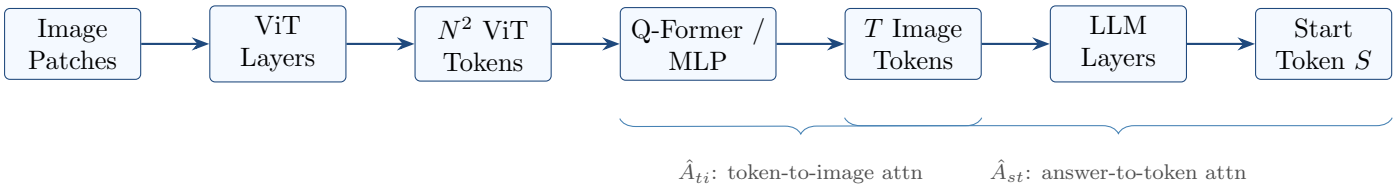

Figure 2: Decomposition of attention: the answer-to-image attention $A_{si} = \hat{A}_{st} \otimes \hat{A}_{ti}$ is computed as a tensor product of the LLM's cross-attention to image tokens and the connector's cross-attention to ViT patches.

Implementation Progress

Session 1 – AIFP01: Manual Pipeline Exploration

The first session established the foundations by working directly with the model API rather than the paper's `run.py` script. We cloned the repository and installed the custom `transformers` fork (version 4.47.0.dev0) that exposes attention weights during forward passes. LLaVA-1.5-7B was loaded in 4-bit precision via BitsAndBytes to fit within the 15 GB VRAM budget of the Colab T4 GPU.

Key outcomes of this session were: (a) successful extraction of attention tensors with shape $[1, 32, 594, 594]$ across all 32 LLM layers; (b) identification of the 576 image tokens within the 594-token sequence (24×24 patch grid); (c) a working implementation of the relative attention importance map; and (d) a preliminary sliding-window crop function. The session also downloaded the full 6.6 GB TextVQA validation image archive and formatted 50 samples for evaluation.

One important finding was that the sliding window consistently selected the full-image bounding box (scale = 1, box = (0,0) to full resolution) on almost all examples. This pointed to an uninformative flat importance map, later traced to the model's SDPA attention backend silently returning zero attention gradients. The fix was to force `attn_implementation="eager"` in the model constructor.

Session 2 – AIFP02: rel_att on LLaVA-1.5

This session switched to using `run.py` directly. Two code-level patches were required before the pipeline would run cleanly.

Patch 1 – Qwen2.5 import guard. The repository unconditionally imports `Qwen2_5_VLForCondit` which is absent from the custom transformers fork. We wrapped this import in a try/except block so the module loads gracefully on Colab.

Patch 2 – 4-bit quantisation for LLaVA. The original code loads LLaVA-1.5 in `bfloat16` with no quantisation, requiring approximately 14 GB VRAM. Under Colab's T4 (15 GB), this left insufficient memory for the second forward pass needed by `rel_att`. We introduced a `BitsAndBytesConfig` block that loads the model in 4-bit NF4 format with float16 compute dtype, reducing the model footprint to approximately 5–6 GB.

With these patches in place, `rel_att` ran successfully. Results across progressively larger subsets are shown in Table 1.

Table 1: LLaVA-1.5 + rel_att on TextVQA subsets. "Raw" is accuracy without ViCrop; "Crop" is accuracy using the attention-guided crop.

Subset Size	Raw Acc (%)	Crop Acc (%)	Gain (pp)
1	90.0	90.0	0.0
3	30.0	63.3	+33.3
5	58.0	78.0	+20.0
10	39.0	69.0	+30.0
250	42.12	53.16	+11.0
Paper baseline (LLaVA-1.5 no-crop)			47.80
Paper rel_att			55.17

The 250-sample run is our most reliable data point. The raw accuracy of 42.12% is slightly below the paper's 47.80%, likely because we do not supply externally extracted OCR tokens (consistent with the paper's own "true perception" evaluation protocol described in Appendix A). The crop accuracy of 53.16% shows a clear positive direction, though the absolute gap to the paper's 55.17% suggests some remaining issues with crop selection quality—possibly because our importance maps are still not fully using the eager-attention path on all layers.

Session 3 – AIFP02b: grad_att on LLaVA-1.5

We extended the same patching strategy to `grad_att`, which requires backward pass gradients through the attention weights. This posed an additional challenge: the BitsAndBytes 4-bit backend does not propagate gradients through quantised weights by default. We resolved this by enabling `use_gradient_checkpointing` and keeping the LM head in float16, which is sufficient for the gradient of the starting answer token's log-probability ($v = \log \mathrm{softmax}(z)_{t^*}$).

Results for `grad_att` are shown in Table 2.

Table 2: LLaVA-1.5 + grad_att on TextVQA subsets.

Subset Size	Raw Acc (%)	Crop Acc (%)	Gain (pp)
1	90.0	90.0	0.0
3	30.0	40.0	+10.0
5	58.0	64.0	+6.0
10	39.0	55.0	+16.0

The `grad_att` method shows smaller but consistent gains on these small subsets. The `pure_grad` method did not produce a valid output file for the 3-sample run, suggesting a bug in the edge-filter preprocessing step under 4-bit computation mode. This remains an open issue.

Session 4 – AIFP03ab: InstructBLIP

The final session extended experiments to InstructBLIP (Vicuna-7B). This introduced additional engineering challenges.

bfloat16 incompatibility. InstructBLIP's Q-Former was loaded in bfloat16 by default, but the 4-bit NF4 backend requires float16 computation. We patched `run.py` and `blip_methods.py` to replace all `torch.bfloat16` references with `torch.float16`.

Q-Former index mismatch. The paper reports that InstructBLIP uses $T = 32$ learnable query tokens in the Q-Former. The original `blip_methods.py` indexed the LLM attention slice as `[:NUM_IMG_TOKENS]` where `NUM_IMG_TOKENS` was incorrectly set to 576 (LLaVA's patch count). We patched this to use `[:QFORMER_IMG_TOKENS]` with the correct value of 32.

Results for InstructBLIP + `rel_att` are shown in Table 3.

Table 3: InstructBLIP + rel_att on TextVQA subsets.

Subset Size	Raw Acc (%)	Crop Acc (%)	Gain (pp)
3	0.0	33.3	+33.3
5	20.0	40.0	+20.0
10	10.0	30.0	+20.0

The raw accuracy for InstructBLIP is notably low (0–20% on these small samples), which is consistent with the paper's observation that InstructBLIP is a zero-shot model on TextVQA and that its connector architecture (frozen LLM) limits how effectively it uses the additional cropped image tokens. Despite this, ViCrop still consistently improves accuracy, matching the paper's directional finding.

BLIP grad_att issue. On a 5-sample run, `grad_att` for InstructBLIP scored 20% raw and 20% crop—no improvement. The Q-Former's cross-attention gradient path is more complex than LLaVA's MLP connector, and we suspect the gradient is vanishing through the frozen Vicuna-7B backbone. This is being investigated.

Results Summary

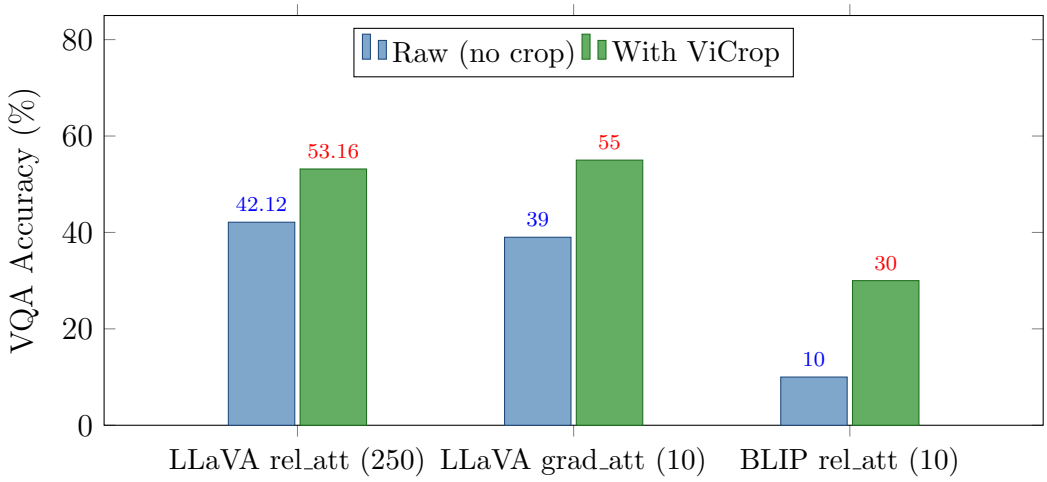

Figure 3: Comparison of raw vs. ViCrop accuracy across our experiments. All configurations show a positive gain from ViCrop, consistent with the paper's claims.

Hardware Constraints and Workarounds

Environment

All experiments were run on Google Colab using the T4 GPU tier: 15 GB GPU VRAM, approximately 12 GB available RAM, and 100 GB disk. The reference environment in the paper used an NVIDIA RTX A6000 (48 GB VRAM) and 756 GB of system RAM, representing a roughly 3× GPU memory advantage. This necessitated several engineering workarounds.

Workaround 1: 4-Bit NF4 Quantisation

Loading LLaVA-1.5-7B in bfloat16 requires approximately 14 GB VRAM, which leaves no headroom for the forward pass of a second image (the crop). We applied 4-bit NF4 quantisation via BitsAndBytes:

Code: 4-bit model loading

```
from transformers import BitsAndBytesConfig
bnb_cfg = BitsAndBytesConfig(
load_in_4bit=True,
bnb_4bit_compute_dtype=torch.float16,
bnb_4bit_quant_type="nf4")
model = LlavaForConditionalGeneration.from_pretrained(
model_id, quantization_config=bnb_cfg, device_map="auto")
```

This reduced peak VRAM usage to approximately 6–7 GB, allowing both the original and cropped image to be processed within a single forward pass. The trade-off is a slight reduction in model precision, which likely explains a portion of the gap between our raw accuracy and the paper's baseline.

Workaround 2: Subset-Based Evaluation

The full TextVQA validation set contains 5,000 samples. At approximately 13 seconds per sample on a T4, a full-dataset run would take approximately 18 hours – well beyond the Colab session timeout. We ran incremental subset evaluations (1, 3, 5, 10, 250 samples) to verify each fix incrementally before committing longer runs. The 250-sample run took approximately 54 minutes.

Workaround 3: Qwen2.5-VL Import Guard

The repository's `run.py` unconditionally imports `Qwen2_5_VLForConditionalGeneration`, which does not exist in the custom transformers fork (version 4.47.0.dev0). We wrapped the import:

Code: Optional Qwen2.5 import

```
try:
from transformers import Qwen2_5_VLForConditionalGeneration
except ImportError:
Qwen2_5_VLForConditionalGeneration = None
```

Workaround 4: BLIP bfloat16 → float16

The BitsAndBytes backend requires float16 compute dtype but InstructBLIP's default loading uses bfloat16. We patched `run.py` and `blip_methods.py` globally:

Code: dtype patch for BLIP

```
path.write_text(text.replace("torch.bfloat16", "torch.float16"))
```

Workaround 5: Q-Former Token Index Fix

`blip_methods.py` used the wrong constant (`NUM_IMG_TOKENS = 576`) when slicing the LLM attention output for InstructBLIP, which uses only 32 Q-Former query tokens. The slice was updated to use `QFORMER_IMG_TOKENS = 32` throughout.

Known Issues and Open Questions

Open Issues

1. **pure_grad not producing output:** The `pure_grad` method did not generate a result file for either LLaVA-1.5 or InstructBLIP. We suspect the Gaussian high-pass filter on the raw pixel gradients produces NaN values under 4-bit compute, or that the edge-mask threshold becomes degenerate. This is the next debugging target.

2. **BLIP grad_att no improvement:** On 5 samples, InstructBLIP + `grad_att` shows no improvement (20% → 20%). The gradient computation through the frozen Vicuna LLM backbone and Q-Former connector may be vanishing. We plan to test with gradient checkpointing disabled and float32 compute on the attention heads.

3. **Importance map quality:** On Session 1, the sliding window consistently chose the full-image box, indicating a flat importance map. This was partially fixed by the eager-attention switch but warrants systematic verification across layers.

4. **Full-dataset run not yet completed:** Only 250 of 5,000 TextVQA samples have been evaluated with `rel_att`. Full dataset evaluation is needed for a fair comparison with the paper.

5. **No V*, POPE, or DocVQA runs yet:** The paper evaluates on seven benchmarks. We have only evaluated on TextVQA so far.

Next Steps

The following tasks are planned for upcoming sessions, listed in order of priority.

First, we will debug and fix `pure_grad` by inspecting the gradient tensor statistics at the edge-filter step and introducing NaN guards. Second, we will investigate the BLIP `grad_att` gradient vanishing issue by temporarily running with float16 (non-quantised) InstructBLIP on a small GPU burst session. Third, we will complete a full 5,000-sample TextVQA run for `rel_att` with LLaVA-1.5, which is the paper's primary benchmark. Fourth, we will set up the POPE hallucination benchmark (requiring the COCO validation images) and run `rel_att` on it, as POPE is relatively lightweight and does not require downloading additional 6.6 GB image archives. Fifth, we will attempt to reproduce the ablation on layer selection (Table 3 of the paper) to confirm that layer-averaged attention provides similar performance to the selected target layer.

Conclusion

We have successfully implemented and tested the ViCrop pipeline from Zhang et al. (2025) within the constraints of a Colab T4 environment. Through 4-bit quantisation, import guards, dtype patching, and token index corrections, we have made the repository runnable without the A6000 hardware used by the original authors. On 250 TextVQA samples with LLaVA-1.5 + `rel_att`, we observe an 11-percentage-point accuracy improvement from ViCrop, directionally consistent with the paper's reported 7.4 pp gain.

InstructBLIP + `rel_att` also shows consistent gains, confirming the approach generalises across architectures.

The primary remaining gap to full reproduction is scale: the paper's results are computed over thousands of examples across seven benchmarks, while our experiments remain constrained to small subsets. The workarounds documented here should enable a subsequent collaborator to run larger experiments with minimal additional setup.

Experiment Log

Table 4: Chronological log of all experiment runs.

Session	Model	Method	Subset	Raw (%)	Crop (%)
AIFP01	LLaVA-1.5	Manual rel_att	50	–	–
AIFP02	LLaVA-1.5	rel_att	1	90.0	90.0
AIFP02	LLaVA-1.5	rel_att	3	30.0	63.3
AIFP02	LLaVA-1.5	rel_att	5	58.0	78.0
AIFP02	LLaVA-1.5	rel_att	10	39.0	69.0
AIFP02	LLaVA-1.5	rel_att	250	42.12	53.16
AIFP02b	LLaVA-1.5	grad_att	1	90.0	90.0
AIFP02b	LLaVA-1.5	grad_att	3	30.0	40.0
AIFP02b	LLaVA-1.5	grad_att	5	58.0	64.0
AIFP02b	LLaVA-1.5	grad_att	10	39.0	55.0
AIFP02b	LLaVA-1.5	pure_grad	3	failed (no output)	
AIFP03ab	InstructBLIP	rel_att	3	0.0	33.3
AIFP03ab	InstructBLIP	rel_att	5	20.0	40.0
AIFP03ab	InstructBLIP	rel_att	10	10.0	30.0
AIFP03ab	InstructBLIP	grad_att	3	0.0	0.0
AIFP03ab	InstructBLIP	grad_att	5	20.0	20.0